

# Phân tích thuật toán Nén dữ liệu

## Huffman Coding & Lempel-Ziv 77 (LZ77)

Hoàng Phúc - Đăng Khoa

Khoa Khoa học Máy tính  
Trường Đại học Công nghệ Thông tin (UIT)  
ĐHQG-HCM

Ngày 3 tháng 6 năm 2026

# Nội dung trình bày

- 1 Bài toán nén dữ liệu
- 2 Thuật toán Huffman Coding
- 3 Thuật toán LZ77
- 4 Ưu nhược và ứng dụng
- 5 Tổng kết & Tài liệu tham khảo

# Mục lục

- 1 Bài toán nén dữ liệu
- 2 Thuật toán Huffman Coding
- 3 Thuật toán LZ77
- 4 Ưu nhược và ứng dụng
- 5 Tổng kết & Tài liệu tham khảo

# Bài toán nén dữ liệu

**Mục tiêu chung:** biểu diễn dữ liệu bằng ít bit hơn nhưng vẫn có thể khôi phục lại dữ liệu ban đầu.

## Input

Một chuỗi dữ liệu ban đầu:

$$S = s_1 s_2 \dots s_N$$

trong đó mỗi  $s_i$  là một ký tự hoặc ký hiệu trong tập ký hiệu nguồn  $\Sigma$ .

## Output

Một biểu diễn nén  $C$  sao cho kích thước tính theo bit của  $C$  nhỏ hơn kích thước ban đầu:

$$\text{size}(C) < \text{size}(S)$$

## Định nghĩa kích thước

Trong bài toán nén dữ liệu, kích thước của dữ liệu được đo bằng số bit cần để biểu diễn dữ liệu đó.

$$\text{size}(X) = \text{số bit cần để lưu } X$$

## Yêu cầu đối với nén không mất dữ liệu

Dữ liệu sau khi giải nén phải khôi phục chính xác dữ liệu ban đầu:

$$\text{Decompress}(C) = S$$

# Hai hướng tiếp cận trong bài seminar

Trong bài seminar này, ta xét hai thuật toán nén không mất dữ liệu:

| Thuật toán            | Ý tưởng chính                                                              | Dạng dữ liệu khai thác               |
|-----------------------|----------------------------------------------------------------------------|--------------------------------------|
| <b>Huffman Coding</b> | Gán mã ngắn cho ký tự xuất hiện nhiều và mã dài hơn cho ký tự xuất hiện ít | Tần suất xuất hiện của từng ký tự    |
| <b>LZ77</b>           | Thay thế chuỗi lặp lại bằng tham chiếu về vị trí đã xuất hiện trước đó     | Cấu trúc lặp lại trong chuỗi dữ liệu |

# Mục lục

- 1 Bài toán nén dữ liệu
- 2 Thuật toán Huffman Coding**
- 3 Thuật toán LZ77
- 4 Ưu nhược và ứng dụng
- 5 Tổng kết & Tài liệu tham khảo

# Bài toán mã hoá ký tự trong Huffman Coding

## Input:

- Một tập ký tự:

$$\Sigma = \{s_1, s_2, \dots, s_n\}$$

- Tần suất hoặc số lần xuất hiện của mỗi ký tự:

$$p_1, p_2, \dots, p_n$$

## Output:

- Một bảng mã nhị phân cho các ký tự.
- Bộ mã phải là **prefix code** để giải mã không nhập nhằng.



# Mã cố định độ dài: phương án cơ sở

Ở slide trước, ta cần xây dựng một bảng mã nhị phân cho các ký tự. Cách đơn giản nhất là gán cho mỗi ký tự cùng một số bit.

**Mã cố định độ dài** (fixed-length encoding) gán cùng một số bit cho mọi ký tự.

Với tập ký tự:  $\{A, B, C, D, E, F\}$ , ta có số ký tự khác nhau là:  $n = 6$

Số bit tối thiểu cần dùng cho mỗi ký tự là:

$$\lceil \log_2 n \rceil = \lceil \log_2 6 \rceil = 3 \text{ bits}$$

Một bảng mã cố định độ dài có thể là:

| Ký tự    | A    | B    | C    | D    | E    | F    |
|----------|------|------|------|------|------|------|
| Tần suất | 0.45 | 0.13 | 0.12 | 0.16 | 0.09 | 0.05 |
| Mã       | 000  | 001  | 010  | 011  | 100  | 101  |

# Hạn chế của mã cố định độ dài

Một bảng mã cố định độ dài có thể là:

| Ký tự    | A    | B    | C    | D    | E    | F    |
|----------|------|------|------|------|------|------|
| Tần suất | 0.45 | 0.13 | 0.12 | 0.16 | 0.09 | 0.05 |
| Mã       | 000  | 001  | 010  | 011  | 100  | 101  |

## Vấn đề

Ký tự xuất hiện nhiều và ký tự xuất hiện ít đều dùng cùng số bit. Do đó, mã cố định độ dài không tận dụng được tần suất xuất hiện của dữ liệu.

## Ý tưởng của Huffman Coding:

Ký tự xuất hiện nhiều  $\Rightarrow$  mã ngắn hơn

Ký tự xuất hiện ít  $\Rightarrow$  có thể dùng mã dài hơn

# Mã biến độ dài

**Mã biến độ dài** (*variable-length encoding*) cho phép mỗi ký tự có độ dài mã khác nhau.

Thay vì mọi ký tự đều dùng 3 bits như mã cố định độ dài, ta có thể gán:

| Ký tự    | <i>A</i> | <i>B</i> | <i>C</i> | <i>D</i> | <i>E</i> | <i>F</i> |
|----------|----------|----------|----------|----------|----------|----------|
| Tần suất | 0.45     | 0.13     | 0.12     | 0.16     | 0.09     | 0.05     |
| Mã       | 0        | 01       | 10       | 111      | 110      | 100      |

Tuy nhiên, mã biến độ dài có thể gây nhập nhằng trong quá trình giải mã nếu thiết kế không cẩn thận.

# Vấn đề giải mã nhập nhằng

Với bảng mã ở slide trước, hãy xét chuỗi bit:

0110

Chuỗi bit này có thể được tách theo nhiều cách khác nhau:

$0|110 \rightarrow AE$       hoặc       $01|10 \rightarrow BC$

Cùng một chuỗi bit nhưng có thể dẫn đến nhiều kết quả giải mã khác nhau.

⇒ Không đảm bảo **tính giải mã duy nhất** (*unique decodability*).

## Vấn đề cần giải quyết

Làm sao thiết kế mã biến độ dài sao cho mỗi chuỗi bit chỉ có đúng một cách giải mã?

# Prefix Code

**Prefix code** là bộ mã trong đó không có mã của ký tự nào là tiền tố của mã ký tự khác.

Ví dụ bộ mã không thỏa prefix code:

| Ký tự | A | B  | C  | D   | E   | F   |
|-------|---|----|----|-----|-----|-----|
| Mã    | 0 | 01 | 10 | 111 | 110 | 100 |

Ví dụ bộ mã thỏa prefix code:

| Ký tự | A | B   | C   | D   | E    | F    |
|-------|---|-----|-----|-----|------|------|
| Mã    | 1 | 010 | 011 | 000 | 0010 | 0011 |

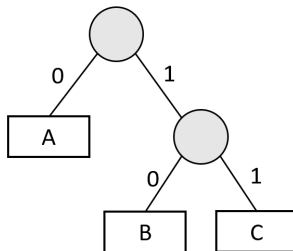
# Biểu diễn Prefix Code bằng cây nhị phân (1)

Mỗi bộ **prefix code** có thể được biểu diễn bằng một **cây nhị phân**.

- Mỗi ký tự được đặt tại một **nút lá**.
- Cạnh đi sang trái được gán nhãn 0.
- Cạnh đi sang phải được gán nhãn 1.
- Mã của một ký tự là chuỗi bit trên đường đi từ gốc đến nút lá chứa ký tự đó.

Ví dụ:

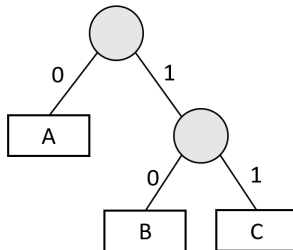
| Ký tự | A | B  | C  |
|-------|---|----|----|
| Mã    | 0 | 10 | 11 |



# Biểu diễn Prefix Code bằng cây nhị phân (2)

Ví dụ:

| Ký tự | A | B  | C  |
|-------|---|----|----|
| Mã    | 0 | 10 | 11 |



## Nhận xét

Ký tự nằm càng gần gốc thì mã càng ngắn. Ký tự nằm càng xa gốc thì mã càng dài.

**Bài toán Huffman:** xây cây sao cho ký tự xuất hiện nhiều nằm gần gốc hơn.

# Mục tiêu tối ưu của Huffman Coding

Sau khi biểu diễn mã bằng cây nhị phân, độ dài mã  $l_i$  chính bằng độ sâu của ký tự  $i$  trong cây.

Giả sử:

- $p_i$  = tần suất xuất hiện của ký tự  $i$ .
- $l_i$  = độ dài mã của ký tự  $i$ .

Khi đó, số bit trung bình cần dùng để mã hoá một ký tự là:

$$L = \sum_{i=1}^n p_i \cdot l_i$$

Mục tiêu của Huffman Coding là:

$$\min \sum_{i=1}^n p_i \cdot l_i$$



# Chiến lược tham lam của Huffman Coding

## Ý tưởng chính

Hai ký tự có tần suất nhỏ nhất nên được đặt ở vị trí sâu nhất trong cây.

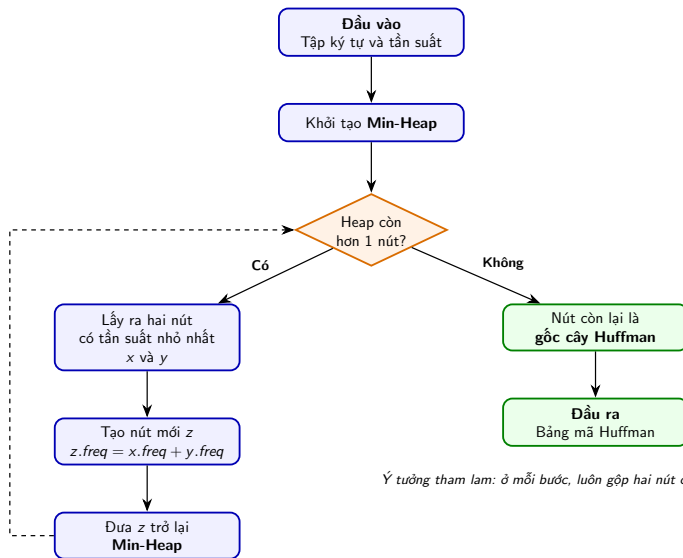
Do đó, Huffman Coding chọn chiến lược:

- 1 Chọn hai ký tự hoặc hai nút có tần suất nhỏ nhất.
- 2 Gộp chúng thành một nút cha.
- 3 Tần suất của nút cha bằng tổng tần suất hai nút con.
- 4 Xem nút cha này như một nút mới và tiếp tục lặp lại.

## Ý tưởng tham lam

Ở mỗi bước, Huffman đưa hai phần tử ít xuất hiện nhất xuống sâu hơn trong cây, để dành các mã ngắn hơn cho những phần tử xuất hiện nhiều hơn.

# Sơ đồ xây dựng cây Huffman



Ý tưởng tham lam: ở mỗi bước, luôn gộp hai nút có tần suất nhỏ nhất.

# Ví dụ 1: Xây cây Huffman

**VÍ DỤ 1:** Xét bảng chữ cái gồm 6 ký tự  $\{A, B, C, D, E, F\}$  với các tần suất xuất hiện sau trong một văn bản được tạo thành từ các ký tự này:

| Ký tự    | <i>A</i> | <i>B</i> | <i>C</i> | <i>D</i> | <i>E</i> | <i>F</i> |
|----------|----------|----------|----------|----------|----------|----------|
| Tần suất | 0.45     | 0.13     | 0.12     | 0.16     | 0.09     | 0.05     |

|      |
|------|
| 0.05 |
| F    |

|      |
|------|
| 0.09 |
| E    |

|      |
|------|
| 0.12 |
| C    |

|      |
|------|
| 0.13 |
| B    |

|      |
|------|
| 0.16 |
| D    |

|      |
|------|
| 0.45 |
| A    |

# Bước 1: Gộp F(0.05) và E(0.09) $\rightarrow$ 0.14

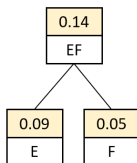
|      |
|------|
| 0.12 |
| C    |

|      |
|------|
| 0.13 |
| B    |

|      |
|------|
| 0.14 |
| EF   |

|      |
|------|
| 0.16 |
| D    |

|      |
|------|
| 0.45 |
| A    |



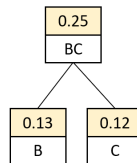
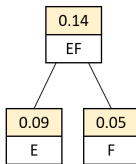
## Bước 2: Gộp C(0.12) và B(0.13) $\rightarrow$ 0.25

|      |
|------|
| 0.14 |
| EF   |

|      |
|------|
| 0.16 |
| D    |

|      |
|------|
| 0.25 |
| BC   |

|      |
|------|
| 0.45 |
| A    |

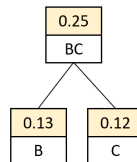
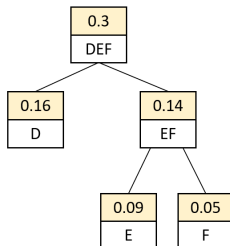


### Bước 3: Gộp 0.14 và D(0.16) $\rightarrow$ 0.30

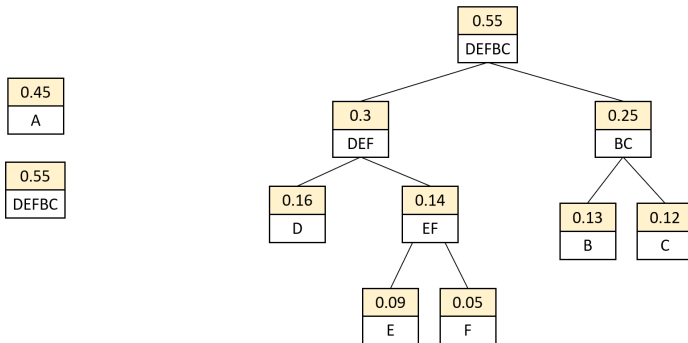
|      |
|------|
| 0.25 |
| BC   |

|     |
|-----|
| 0.3 |
| DEF |

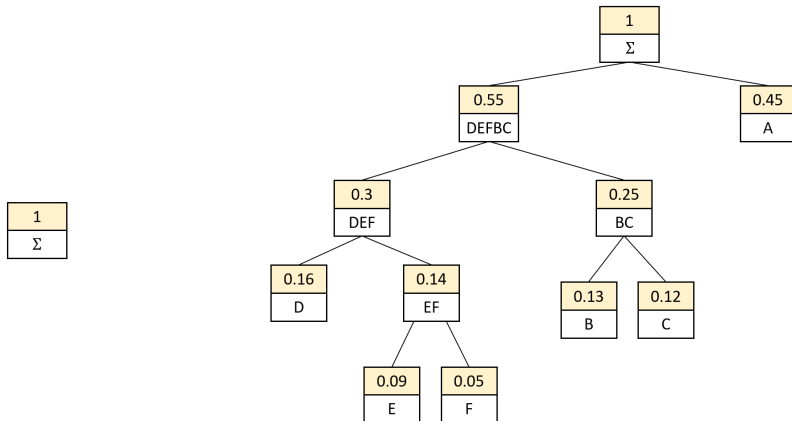
|      |
|------|
| 0.45 |
| A    |



## Bước 4: Gộp 0.25 và 0.30 $\rightarrow$ 0.55

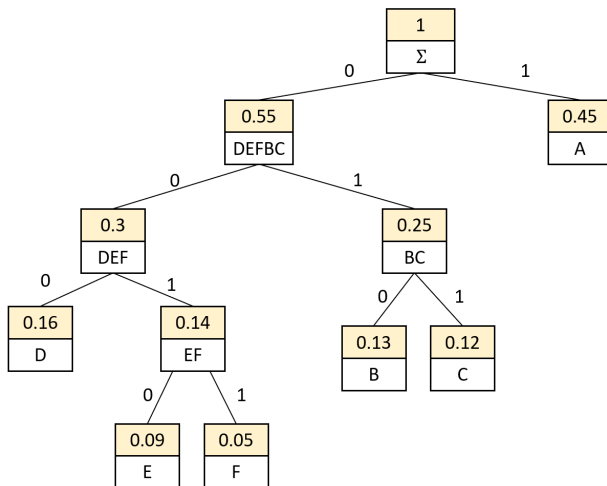


## Bước 5: Gộp A(0.45) và 0.55 $\rightarrow$ 1.00



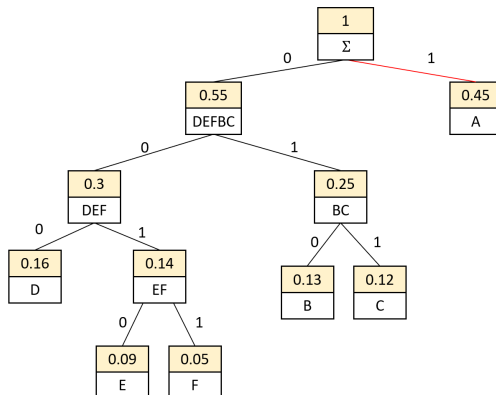


# Ví dụ 1: Cây Huffman hoàn chỉnh



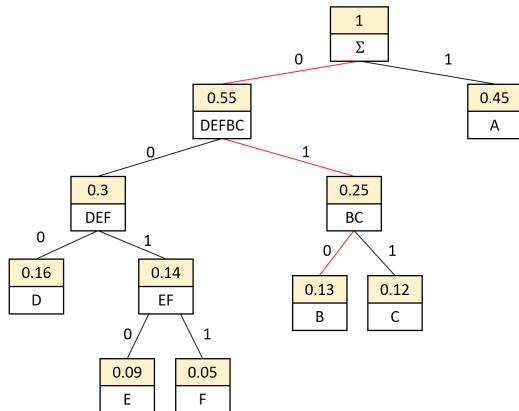
# Ví dụ 1: Suy ra mã Huffman từ cây

| Ký tự      | A    | B    | C    | D    | E    | F    |
|------------|------|------|------|------|------|------|
| Tần suất   | 0.45 | 0.13 | 0.12 | 0.16 | 0.09 | 0.05 |
| Mã Huffman | 1    |      |      |      |      |      |



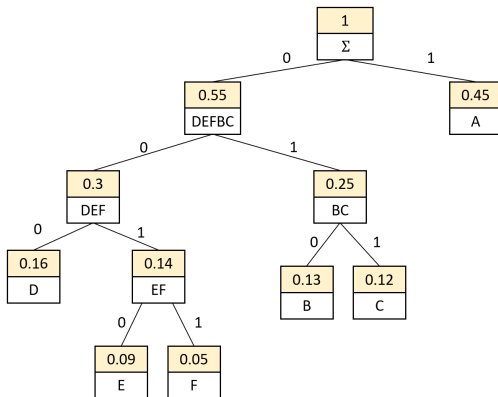
# Ví dụ 1: Suy ra mã Huffman từ cây

| Ký tự    | <i>A</i> | <i>B</i> | <i>C</i> | <i>D</i> | <i>E</i> | <i>F</i> |
|----------|----------|----------|----------|----------|----------|----------|
| Tần suất | 0.45     | 0.13     | 0.12     | 0.16     | 0.09     | 0.05     |
| Mã       | 1        | 010      |          |          |          |          |



# Ví dụ 1: Bảng mã Huffman thu được

| Ký tự      | A    | B    | C    | D    | E    | F    |
|------------|------|------|------|------|------|------|
| Tần suất   | 0.45 | 0.13 | 0.12 | 0.16 | 0.09 | 0.05 |
| Mã Huffman | 1    | 010  | 011  | 000  | 0010 | 0011 |



## Ví dụ 2: Xây cây Huffman từ một chuỗi cụ thể

**VÍ DỤ 2:** Xét chuỗi sau:

“ADBADEDBBDD”

Ta dễ dàng lập được bảng sau:

| Ký tự            | A | B | D | E |
|------------------|---|---|---|---|
| Số lần xuất hiện | 2 | 3 | 5 | 1 |

|   |
|---|
| 1 |
| E |

|   |
|---|
| 2 |
| A |

|   |
|---|
| 3 |
| B |

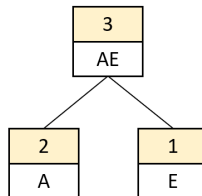
|   |
|---|
| 5 |
| D |

# Bước 1: Gộp E(1) và A(2) $\rightarrow$ 3

|    |
|----|
| 3  |
| AE |

|   |
|---|
| 3 |
| B |

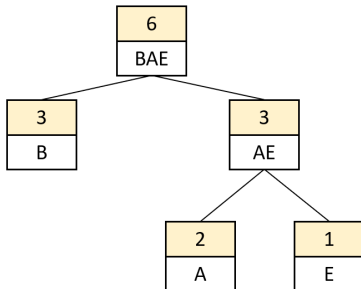
|   |
|---|
| 5 |
| D |



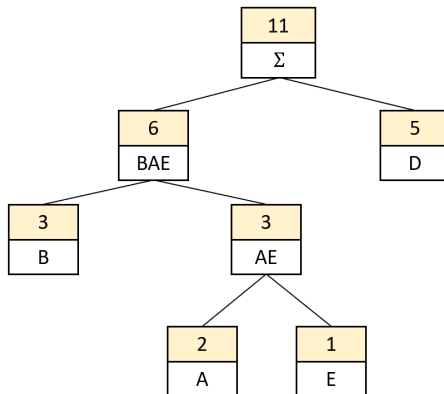
## Bước 2: Gộp 3 và B(3) $\rightarrow$ 6

|   |
|---|
| 5 |
| D |

|     |
|-----|
| 6   |
| BAE |

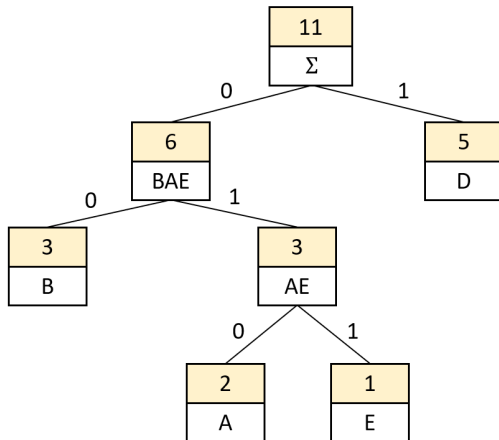


### Bước 3: Gộp D(5) và 6 $\rightarrow$ 11



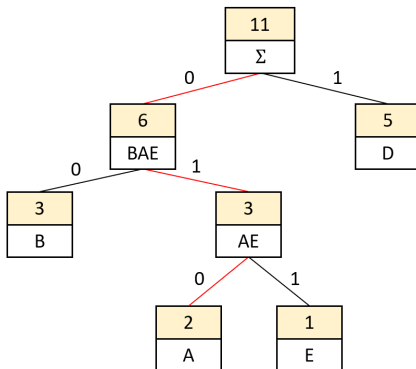


## Ví dụ 2: Cây Huffman hoàn chỉnh



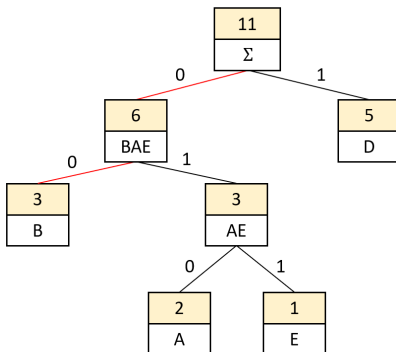
## Ví dụ 2: Suy ra mã Huffman từ cây

| Ký tự            | <i>A</i> | <i>B</i> | <i>D</i> | <i>E</i> |
|------------------|----------|----------|----------|----------|
| Số lần xuất hiện | 2        | 3        | 5        | 1        |
| Mã Huffman       | 010      |          |          |          |



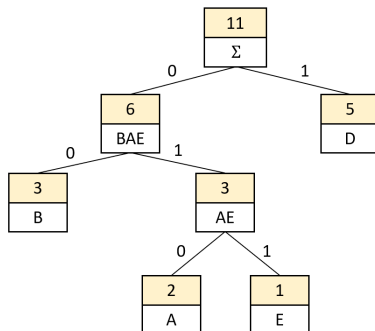
## Ví dụ 2: Suy ra mã Huffman từ cây

| Ký tự      | <i>A</i> | <i>B</i> | <i>D</i> | <i>E</i> |
|------------|----------|----------|----------|----------|
| Tần suất   | 2        | 3        | 5        | 1        |
| Mã Huffman | 010      | 00       | 00       | 1        |



## Ví dụ 2: Bảng mã Huffman thu được

| Ký tự            | <i>A</i> | <i>B</i> | <i>D</i> | <i>E</i> |
|------------------|----------|----------|----------|----------|
| Số lần xuất hiện | 2        | 3        | 5        | 1        |
| Mã Huffman       | 010      | 00       | 1        | 011      |



# Giải mã Huffman

Sau khi mã hóa dữ liệu bằng cây Huffman, việc giải mã các chuỗi bit trở nên đơn giản.

## Quá trình giải mã:

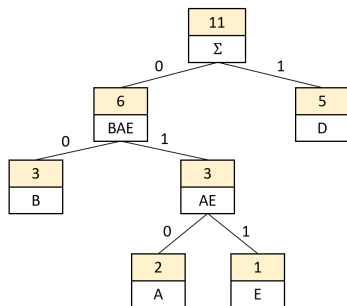
- 1 Đọc chuỗi bit từ trái sang phải.
- 2 Bắt đầu từ gốc của cây Huffman.
- 3 Nếu gặp bit 0, đi sang nhánh trái.
- 4 Nếu gặp bit 1, đi sang nhánh phải.
- 5 Khi gặp lá, ta xác định được ký tự tương ứng và quay lại gốc cây để giải mã ký tự tiếp theo.
- 6 Lặp lại cho đến khi giải mã hết tất cả các ký tự trong chuỗi bit.

## Lưu ý

Vì Huffman là **prefix code**, ta luôn biết khi nào đã đến lá (ký tự). Điều này đảm bảo rằng quá trình giải mã là duy nhất và không nhập nhằng.

# Ví dụ giải mã Huffman

| Ký tự      | A   | B  | D | E   |
|------------|-----|----|---|-----|
| Tần suất   | 2   | 3  | 5 | 1   |
| Mã Huffman | 010 | 00 | 1 | 011 |



Chuỗi bit cần giải mã:

0 1 0 0 0 1 0 0

↓ ↓ ↓ ↓

**A B D B**

Kết quả giải mã: **ABDB**

# Tỉ lệ nén (Compression Ratio)

**Tỉ lệ nén** dùng để đánh giá dữ liệu được nén nhỏ đi bao nhiêu lần so với ban đầu.

$$\text{Compress Ratio} = \frac{\text{Uncompressed Size}}{\text{Compressed Size}}$$

- Miền giá trị:  $CR \geq 1$
- $CR$  càng lớn thì hiệu quả nén càng cao

Với mã Huffman ở ví dụ 1, tỉ lệ nén là 1.34, tức là dữ liệu ban đầu lớn gấp 1.34 lần dữ liệu sau nén.

Với mã Huffman ở ví dụ 2, tỉ lệ nén là 1.1, tức là dữ liệu ban đầu lớn gấp 1.1 lần dữ liệu sau nén.

## Nhận xét:

- Nếu tần suất các ký tự gần đều nhau, hiệu quả nén thấp.
- Nếu một số ký tự xuất hiện nhiều hơn rõ rệt, hiệu quả nén cao hơn.

# Kết luận phần Huffman Coding

**Huffman Coding** là thuật toán nén không mất dữ liệu dựa trên tần suất xuất hiện của ký tự.

- Ký tự xuất hiện nhiều được gán **mã ngắn**.
- Ký tự xuất hiện ít được gán **mã dài hơn**.
- Bộ mã Huffman là **prefix code**, nên quá trình giải mã là duy nhất.
- Thuật toán xây cây bằng chiến lược **tham lam**: luôn gộp hai nút có tần suất nhỏ nhất.
- Có thể cài đặt hiệu quả bằng **Priority Queue / Min-Heap**.

## Ý nghĩa

Huffman Coding tối ưu hóa độ dài mã dựa trên tần suất xuất hiện, từ đó giảm số bit trung bình cần dùng cho mỗi ký tự.



# Mục lục

- 1 Bài toán nén dữ liệu
- 2 Thuật toán Huffman Coding
- 3 Thuật toán LZ77**
- 4 Ưu nhược và ứng dụng
- 5 Tổng kết & Tài liệu tham khảo

# Lịch sử hình thành & Khái niệm

- **Lịch sử:** Được công bố bởi Abraham Lempel và Jacob Ziv vào năm **1977** [2].
- **Khái niệm:** Thuật toán nén dựa trên từ điển (*Dictionary-based*), nhưng sử dụng chính dữ liệu đã đọc làm từ điển [3].

## Nguyên lý chính

Thay thế các cụm ký tự đã xuất hiện bằng một tham chiếu đến vị trí của nó trong quá khứ.

# Khái niệm: Nén từ điển & Tham chiếu

## 1. Từ điển thông thường (Dictionary)

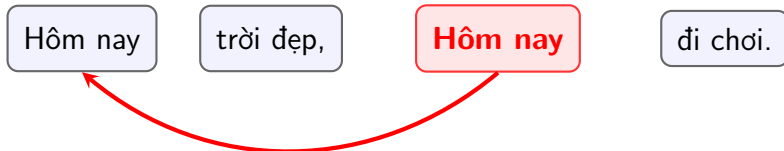
Phải có một bảng quy ước tạo sẵn: **VN** = "*Việt Nam*".

→ Gặp chữ **VN**, máy tính phải mở bảng mã ra tra.

## 2. Tham chiếu quá khứ (Cách của LZ77)

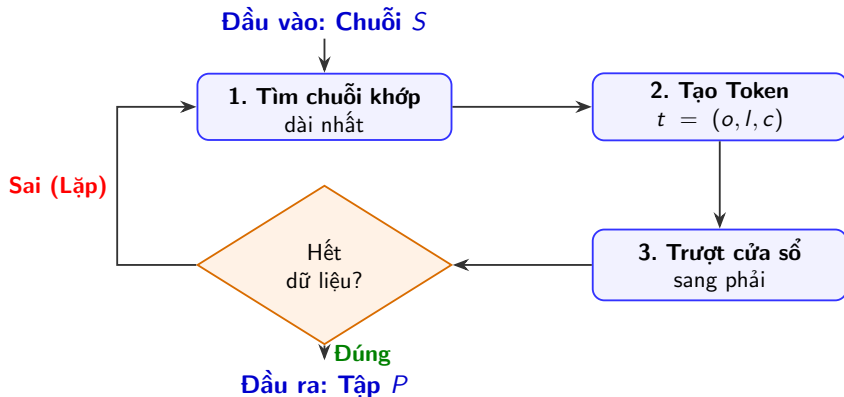
**Không cần bảng mã!** Nó tự biến đoạn văn vừa đọc thành từ điển.

**Tham chiếu (Reference)** đơn giản là một mũi tên: "*Lùi lại X bước, chép Y chữ*".



**Tham chiếu:** Trở về quá khứ để chép lại

# Quy trình hoạt động (Compression Pipeline)



## Định dạng Output (Tập $P$ )

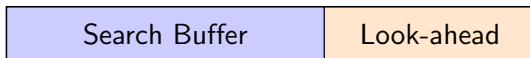
$$P = \{(o_1, l_1, c_1), (o_2, l_2, c_2), \dots, (o_n, l_n, c_n)\}$$

Trong đó:  $o$  = Offset,  $l$  = Length,  $c$  = ký tự tiếp theo.

# Mô tả ý tưởng: Cửa sổ trượt

**Cơ chế:** Sử dụng "Cửa sổ trượt" (*Sliding Window*) để tìm kiếm các chuỗi lặp lại. Cửa sổ được chia làm 2 phần:

- 1 **Search Buffer (Bộ đệm tìm kiếm):** Chứa dữ liệu đã xử lý.
- 2 **Look-ahead Buffer (Bộ đệm nhìn trước):** Chứa dữ liệu chờ nén.



↓ Trượt sang phải dựa trên  $(l + 1)$  của Token ↓



# Ví dụ chạy tay LZ77 (Khởi động)

**Chuỗi đầu vào:** abababa (Search Window = 4, look-ahead = 7)

| Bước | Search Buffer | Look-ahead | Token $(o, l, c)$ |
|------|---------------|------------|-------------------|
| 1    | (trống)       | abababa    | $(0, 0, a)$       |
| 2    | a             | bababa     | $(0, 0, b)$       |
| 3    | ab            | ababa      | $(2, 4, a)$       |
| 4    | abababa       | (hết)      | (kết thúc)        |

**Kết quả:**  $P = \{(0, 0, a), (0, 0, b), (2, 4, a)\}$

# Ví dụ chạy tay (Chi tiết)

**Chuỗi đầu vào:** abcabdabcabc (12 ký tự)

**Cấu hình:** Search Window = 6, Look-ahead = 4.

*\*Quy tắc: Độ dài khớp tối đa  $l \leq \text{Look-ahead} - 1$ .*

| B. | Search Buffer | Look-ahead | Token       | Giải thích                                  |
|----|---------------|------------|-------------|---------------------------------------------|
| 1  | (trống)       | abca       | $(0, 0, a)$ | Không có dữ liệu cũ $\rightarrow$ xuất 'a'. |
| 2  | a             | bcab       | $(0, 0, b)$ | 'b' chưa xuất hiện $\rightarrow$ xuất 'b'.  |
| 3  | ab            | cabd       | $(0, 0, c)$ | 'c' chưa xuất hiện $\rightarrow$ xuất 'c'.  |
| 4  | abc           | abda       | $(3, 2, d)$ | Khớp <b>ab</b> .                            |
| 5  | abcabd        | abca       | $(6, 3, a)$ | Khớp <b>abc</b> .                           |
| 6  | bdabca        | bc         | $(3, 1, c)$ | Khớp <b>b</b> .                             |

**Kết quả:**  $P = \{(0, 0, a), (0, 0, b), (0, 0, c), (3, 2, d), (6, 3, a), (3, 1, c)\}$

# Giải mã LZ77 (Decoding)

**Input (Tập P):**  $\{(0, 0, a), (0, 0, b), (0, 0, c), (3, 2, d), (6, 3, a), (3, 1, c)\}$

**Output:** Khôi phục lại chuỗi abcabdabcabc.

| Bước | Token $(o, l, c)$ | Thao tác & Chuỗi hiện tại                             |
|------|-------------------|-------------------------------------------------------|
| 1    | $(0, 0, a)$       | Xuất 'a' $\rightarrow$ a                              |
| 2    | $(0, 0, b)$       | Xuất 'b' $\rightarrow$ ab                             |
| 3    | $(0, 0, c)$       | Xuất 'c' $\rightarrow$ abc                            |
| 4    | $(3, 2, d)$       | Lùi 3 chép 2 (ab), thêm 'd' $\rightarrow$ abcabd      |
| 5    | $(6, 3, a)$       | Lùi 6 chép 3 (abc), thêm 'a' $\rightarrow$ abcabdabca |
| 6    | $(3, 1, c)$       | Lùi 3 chép 1 (b), thêm 'c' $\rightarrow$ abcabdabcabc |



# Mục lục

- 1 Bài toán nén dữ liệu
- 2 Thuật toán Huffman Coding
- 3 Thuật toán LZ77
- 4 Ưu nhược và ứng dụng
- 5 Tổng kết & Tài liệu tham khảo

# So sánh ưu nhược điểm

| Tiêu chí        | Huffman Coding          | LZ77                                |
|-----------------|-------------------------|-------------------------------------|
| Độ phức tạp nén | $O(N \log k)$           | $O(N \cdot W)$ ( $W$ : window size) |
| Điểm mạnh       | Nén tốt ở mức ký tự     | Khử trùng lặp chuỗi cực tốt         |
| Điểm yếu        | Tồn chi phí lưu bảng mã | Tốn CPU để tìm kiếm chuỗi           |

# Ứng dụng thực tế

**Tại sao chúng ta học cả hai?** Trong thực tế, các định dạng phổ biến sử dụng thuật toán lai (**Hybrid**):

- **Thuật toán DEFLATE [4]:** Kết hợp LZ77 để loại bỏ chuỗi lặp, sau đó dùng Huffman để nén các Token đầu ra.
- **Định dạng File:** .ZIP, .GZIP, .PNG.
- **Truyền tải Web:** Giao thức HTTP sử dụng Gzip/Brotli để nén dữ liệu trước khi gửi qua mạng.

# Mục lục

- 1 Bài toán nén dữ liệu
- 2 Thuật toán Huffman Coding
- 3 Thuật toán LZ77
- 4 Ưu nhược và ứng dụng
- 5 Tổng kết & Tài liệu tham khảo

- **Huffman:** Đại diện cho tư duy tối ưu hóa dựa trên tần suất.
- **LZ77:** Đại diện cho tư duy tối ưu hóa dựa trên cấu trúc và sự lặp lại của dữ liệu.

## Q & A

- [1] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Addison-Wesley, 2011.
- [2] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," in *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337-343, May 1977, doi: 10.1109/TIT.1977.1055714.
- [3] Shirani, S. (2008). Data compression: The complete reference (by d. salomon; 2007)[book review]. *IEEE Signal Processing Magazine*, 25(2), 147-149.
- [4] Deutsch, P. (1996). *DEFLATE compressed data format specification version 1.3* (No. rfc1951).